1. Project Overview

NAGPUR PULSE is a decision-support platform designed to help traffic police identify high-risk locations and efficiently deploy limited police personnel.

The system combines:

- Traffic conditions
- Accident/incident data
- Historical accident data
- Road/junction geography
- Police unit availability
- Police unit locations
- Illegal parking
- Roadworks
- Traffic violations
- Weather
- Public events
- Routing and ETA
- Risk scoring
- Coverage-gap detection
- Personnel optimization

The system answers:

> ***Where should limited traffic-police personnel be deployed right now, why are they needed there, and how should deployment change when conditions change?***

The platform provides recommendations to an authorized operator. It does not autonomously dispatch police.

2. Main Features

- Traffic-risk scoring for Nagpur junctions
- Interactive risk heat map
- High-risk location ranking
- High-risk + unmanned location detection
- Police unit monitoring
- Police unit availability tracking
- Routing and ETA calculation
- Personnel allocation recommendation
- Dynamic redeployment simulation
- Explainable risk scores
- Explainable deployment recommendations
- Manual accept/modify/reject controls
- Baseline vs recommended deployment comparison
- Control-room dashboard

3. Current Data Inventory

Data	Source	Type	Status
Nagpur Junctions	OSM/Nominatim + public map references	Geographic	Available
Traffic	TomTom	Real-time	Implementing
Traffic Incidents	TomTom	Real-time	Implementing
Historical Accidents	nagpur_accidents_2020_2025.xlsx	Simulated historical	Available
Police Units	Police_units_final.csv	Simulated	Available
Illegal Parking	Mockly	Simulated	Available
Routing / ETA	TomTom	Real-time	Implementing
Weather	Weather API	External	Planned
Roadworks	Simulation	Simulated	Planned
Traffic Violations	Simulation	Simulated	Planned
Public Events	Simulation/API	Simulated	Planned
Police GPS	Government/Police system	Secure future data	Future
Official Accident Data	Government/Police system	Secure future data	Future
CCTV	Police CCTV	Secure future data	Future

4. Junction Data

Currently, two JSON datasets provide **40 Nagpur junctions**.

The datasets contain coordinates using WGS84 decimal degrees. Some coordinates are marked approximate and should be verified before safety-critical usage.

CURRENT FIELDS

```
{
  "locationId": "ITWARI_001",
  "name": "Itwari",
  "latitude": 21.1569338,
  "longitude": 79.1102582,
  "type": "JUNCTION",
  "source": "OSM"
}
```

JUNCTION DATASETS

```
data/junctions/
├── nagpur_first_20_junctions.json
└── nagpur_second_20_junctions.json
```

IMPORTANT

Junction IDs should be the common reference used by:

- Traffic
- Incidents
- Police units
- Risk scores

- Coverage
- Recommendations
- Deployments

5. Historical Accident Dataset

FILE

nagpur_accidents_2020_2025.xlsx

The uploaded workbook contains a synthetic/simulated historical accident dataset with **1,823 raw accident records** from 2020-2025.

MAIN FIELDS

AccidentID
Date
Year
Time
Junction
Severity
PrimaryVehicleType
ProbableCause
WeatherCondition
InjuredCount
FatalityCount
VehiclesInvolved
PoliceCaseRegistered

WORKBOOK SECTIONS

README
Summary_ByYear
Summary_ByJunction
Summary_BySeverity
Summary_ByCause
Raw_AccidentLog

USAGE

The dataset is used for:

- Historical accident frequency
- Junction risk features
- Severity analysis
- Cause analysis
- ML training/testing
- Historical baseline

It must remain labeled as **SIMULATED** and should not be presented as official police data.

6. Traffic Data

SOURCE

Tom Tom Traffic API

Traffic data is used to understand current road conditions.

REQUIRED INFORMATION

Current Speed
Free Flow Speed
Travel Time
Free Flow Travel Time
Traffic Delay
Confidence
Road Closure
Timestamp

FLOW

```
Junction / Road Location
            |
            v
TomTom Traffic API
            |
            v
Traffic Adapter
            |
            v
Normalized Traffic Data
            |
            v
Risk Engine
```

INTERNAL EXAMPLE


```
{
   "locationId": "ITWARI_001",
   "currentSpeedKmph": 24,
   "freeFlowSpeedKmph": 40,
   "trafficDelaySeconds": 180,
   "timestamp": "...",
   "source": "TOMTOM"
}
```


7. Accident / Incident Data

CURRENT REAL-TIME SOURCE

TomTom Traffic Incident API.

CURRENT SIMULATED SOURCE

Simulated incidents are used where official accident information is unavailable.

INTERNAL SCHEMA


```
{
   "incidentId": "INC-001",
   "locationId": "ITWARI_001",
   "latitude": 21.1569,
   "longitude": 79.1102,
   "incidentType": "ACCIDENT",
   "severity": "HIGH",
   "timestamp": "...",
   "status": "ACTIVE",
   "source": "SIMULATED"
}
```


SUPPORTED INCIDENT TYPES

ACCIDENT
ROAD_CLOSURE
ROADWORK

```
OBSTRUCTION
LANE_CLOSURE
FLOODING
BROKEN_DOWN_VEHICLE
OTHER
```

Future official police/government incident data can be connected through an adapter using the same schema.

8. Police Unit Data

CURRENT SOURCE

```
https://accident-api.free.beeceptor.com/api/police-units
```

The endpoint provides simulated police-unit data.

CURRENT SIMULATION

20 police units are represented.

Example:


```
{
  "unitId": "NP-U001",
  "unitType": "PATROL",
  "latitude": 21.1508,
  "longitude": 79.1327,
  "status": "AVAILABLE",
  "currentAssignment": null,
  "shift": "DAY",
  "lastUpdated": "2026-08-16T12:44:00+05:30"
}
```


STATUS VALUES

```
AVAILABLE
DEPLOYED
BUSY
OFFLINE
```

ARCHITECTURE

```
Beeceptor
    |
    v
Police Unit API Client
    |
    v
Police Unit Adapter
    |
    v
Police Unit Service
    |
    v
In-memory application state
    |
    v
Frontend
```

No MongoDB is currently required for this module.

9. Illegal Parking Data

CURRENT SOURCE

```
https://mockly.me/custom/api/v1/illegal-parking
```

TYPE

```
SIMULATED
```

Used as a risk feature.

EXAMPLE CONCEPT


```
{
   "locationId": "ITWARI_001",
   "vehicleCount": 12,
   "severity": "HIGH",
   "timestamp": "...",
   "source": "SIMULATED"
}
```


The exact API response should be treated as the source of truth and normalized by the backend.

10. Routing / ETA Data

SOURCE

TomTom Routing API.

PURPOSE

Calculate:

- Road distance
- Estimated travel time
- Traffic delay

between:

```
Police Unit -> Target Junction
```

FLOW

```
Police Unit
       |
       v
Current Coordinates
       |
       v
Routing Service
       |
       v
TomTom Routing API
       |
       v
Distance + ETA
       |
       v
Deployment Optimizer
```

INTERNAL ENDPOINT

```
GET /api/routing/unit/:unitId/to/:junctionId
```

Example:

```
GET /api/routing/unit/NP-U007/to/ITWARI_001
```

RESPONSE


```
{
   "unitId": "NP-U007",
   "destinationId": "ITWARI_001",
   "distanceKm": 3.1,
   "travelTimeMinutes": 6,
   "trafficDelayMinutes": 1,
   "source": "TOMTOM"
}
```


11. Risk Engine

The risk engine combines current and historical information.

INPUTS

Traffic
Incidents
Historical Accidents
Traffic Violations
Illegal Parking
Roadworks
Weather
Public Events
Road Geography

FLOW

```
Data Sources
       |
       v
Normalization
       |
       v
Feature Engineering
       |
       v
Risk Model
       |
       v
Risk Score
       |
       v
Risk Level
```

RISK LEVELS

LOW
MEDIUM
HIGH
CRITICAL

Risk thresholds should remain configurable.

12. ML Service

Recommended implementation:

```
Python
FastAPI
Pandas
NumPy
Scikit-learn / XGBoost
SHAP
```

ARCHITECTURE

```
Node.js Backend
      |
      v
ML API
      |
      v
Feature Engineering
      |
      v
Risk Model
      |
      v
Prediction
      |
      v
Explanation
      |
      v
Risk Score
```

PROTOTYPE APPROACH

Start with:

```
Historical Accident Data
       |
       v
Feature Engineering
       |
       v
XGBoost / Random Forest
       |
       v
Risk Score
```

Advanced traffic prediction can be added later.

13. Explain ability

Every recommendation should provide a reason.

Example:

```
ITWARI JUNCTION

Risk Score: 0.84
Risk Level: HIGH

Main factors:

+ Active accident
+ High traffic delay
+ High historical accident frequency
+ Current coverage gap
- Normal weather
```

The explanation should be based on actual model features.

14. Coverage Engine

The coverage engine determines whether police units are available near a junction.

FLOW

```
Junction
    |
    v
Nearby Police Units
    |
    v
Geographic Distance
    |
    v
Coverage Radius
    |
    v
Coverage Status
```

STATUS

```
COVERED
PARTIALLY_COVERED
UNMANNED
```

KEY FEATURE

```
HIGH RISK
    +
UNMANNED
    =
HIGH PRIORITY
```

15. Deployment Optimization

INPUTS

```
Risk Score
High-risk locations
Police unit status
Police unit locations
Current assignments
Coverage
Travel distance
ETA
Available personnel
```

FLOW

```
Risk Engine
      |
      v
High-Risk Locations
      |
      v
Coverage Engine
      |
      v
Available Units
```

```
       |
       v
Routing / ETA
       |
       v
Optimization
       |
       v
Deployment Recommendation
```

POSSIBLE OPTIMIZER

Google OR-Tools

Objective:

```
      Maximize high-risk coverage
                  +
        Minimize response time
                  +
    Minimize unnecessary movement
                  +
    Respect personnel constraints
```

16. Recommendation System

Example:


```
{
"recommendationId": "REC-001",
"unitId": "NP-U007",
"targetLocationId": "ITWARI_001",
"priority": "CRITICAL",
"etaMinutes": 6,
"reason": [
"High risk",
"Currently unmanned",
"Lowest estimated response time"
],
"status": "PENDING"
}
```


OPERATOR ACTIONS

ACCEPT
MODIFY
REJECT

The system should never automatically dispatch a unit.

17. Dynamic Redeployment

```
Normal Monitoring
       |
       v
New Accident / Incident
       |
       v
Risk Recalculation
       |
       v
Coverage Recalculation
       |
```

```
          v
Available Units
          |
          v
ETA Calculation
          |
          v
Optimization
          |
          v
New Recommendation
          |
          v
Operator Decision
          |
          v
Updated Deployment
```

This is the primary dynamic demonstration for the hackathon.

18. Baseline vs Recommended Deployment

The dashboard should compare:

BASELINE

Current police deployment

RECOMMENDED

NAGPUR PULSE optimized deployment

Compare:

High-risk locations covered
Unmanned high-risk locations
Average ETA
Total travel distance
Unit utilization
Coverage score

19. Frontend Architecture

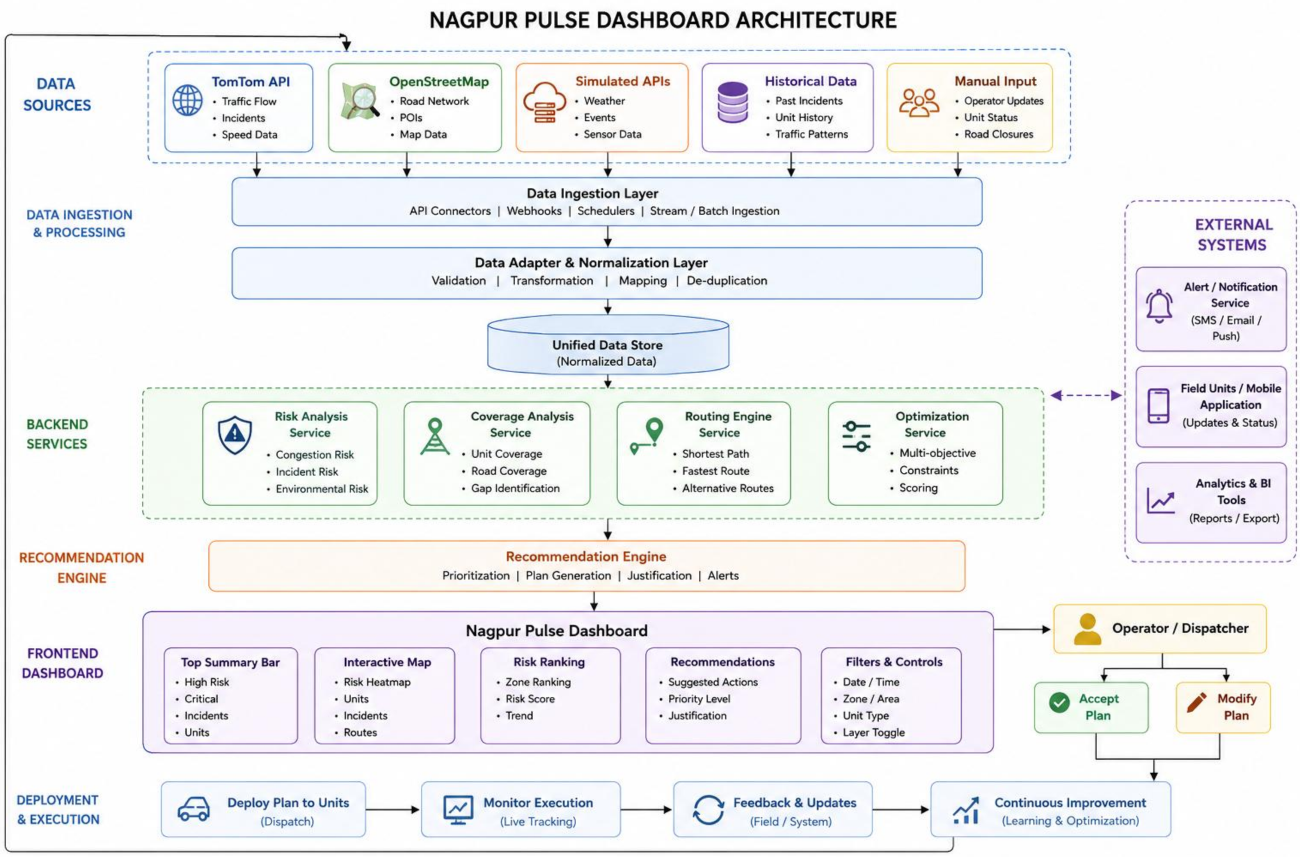
TECHNOLOGY

React
TypeScript
Tailwind CSS
Mapbox / Leaflet
Chart.js / Recharts
Axios

MAIN SCREENS

Dashboard
Risk Heatmap
Incident Monitor
Police Units
Recommendations
Deployment Comparison
Analytics
Simulation

MAIN DASHBOARD



20. Backend Architecture

TECHNOLOGY

Node.js
Express.js
TypeScript / JavaScript
Axios
Zod/Joi
Socket.IO

STRUCTURE

```
backend/
├── routes/
├── controllers/
├── services/
├── adapters/
├── middleware/
├── utils/
└── server.js
```

MAIN SERVICES

TrafficService
IncidentService
PoliceUnitService
RoutingService
CoverageService
RiskService

```
RecommendationService
SimulationService
```

21. Backend API Registry

```
GET    /api/locations

GET    /api/traffic
GET    /api/incidents

GET    /api/police-units
GET    /api/police-units/:unitId
GET    /api/police-units/available

GET    /api/routing/unit/:unitId/to/:junctionId

GET    /api/coverage
GET    /api/coverage/:locationId

GET    /api/risk
GET    /api/risk/:locationId

GET    /api/recommendations

POST /api/recommendations/:id/accept
POST /api/recommendations/:id/reject
PATCH /api/recommendations/:id

GET    /api/deployments

POST /api/simulation/incident
POST /api/simulation/reset
```

22. External API Architecture

Never let the frontend directly depend on external APIs.

```
Frontend
    |
    v
Backend API
    |
    v
External Adapter
    |
    v
External API
```

Examples:

```
TomTomAdapter
BeeceptorPoliceUnitAdapter
MocklyParkingAdapter
WeatherAdapter
OSMAdapter
```

This makes future API replacement easier.

23. Database Strategy

CURRENT PROTOTYPE

No permanent database is required for all modules.

Use:

```
External APIs
+
Static datasets
+
In-memory state
```

FUTURE PRODUCTION

MongoDB can be introduced for:

```
junctions
traffic history
incidents
accidents
police units
deployments
recommendations
audit logs
events
roadworks
violations
```

The frontend and ML service should never directly connect to MongoDB.

24. Future CCTV / Computer Vision

Future feature:

```
CCTV
|
v
Computer Vision
|
v
Accident / Obstruction Detection
|
v
Incident API
|
v
Risk Engine
|
v
Coverage
|
v
Routing
|
v
Optimization
|
v
Recommendation
```

Possible technologies:

```
YOLO
OpenCV
PyTorch
Object Tracking
```

The CV system should generate an incident record rather than directly dispatching a police unit.

25. Real vs Simulated Data

REAL / EXTERNAL

TomTom Traffic
TomTom Traffic Incidents
TomTom Routing
OpenStreetMap

SIMULATED

Police Units
Illegal Parking
Roadworks
Public Events
Traffic Violations
Prototype Incidents

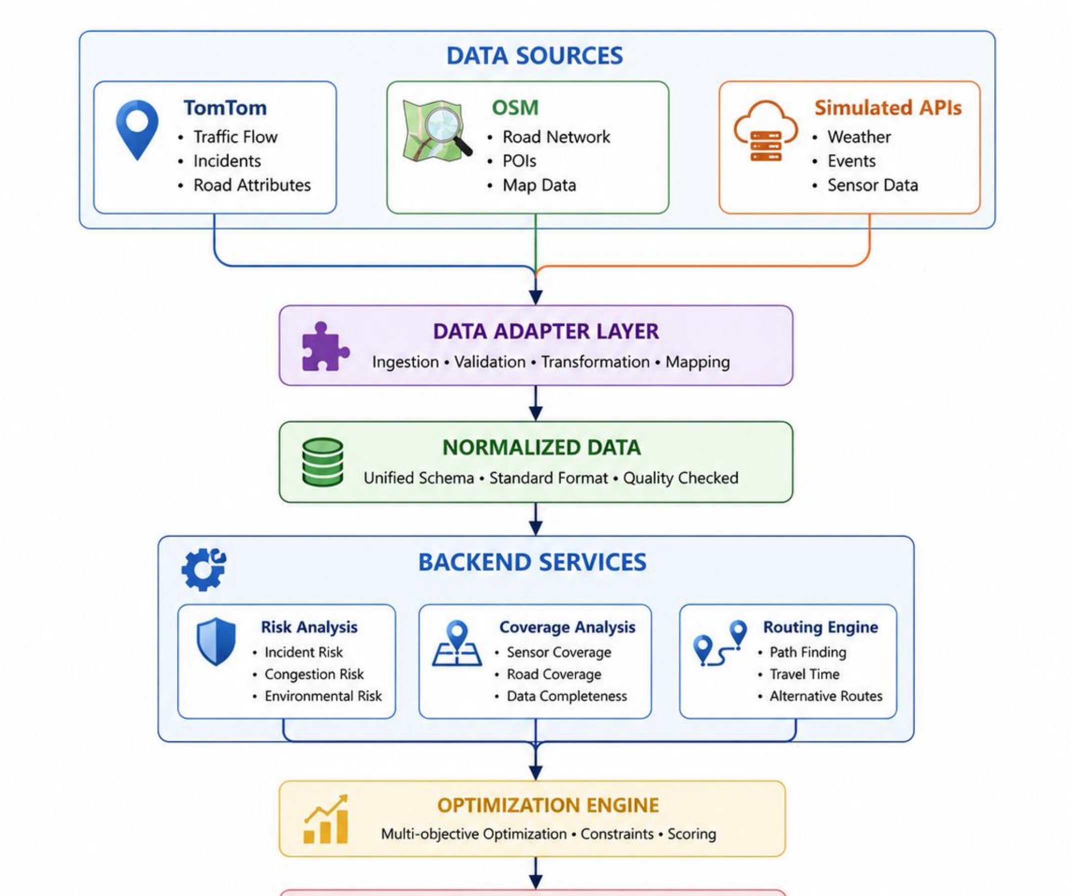
HISTORICAL SYNTHETIC

Nagpur Accident Dataset 2020-2025

FUTURE SECURE

Official Police Accident Data
Police GPS
Police Dispatch Data
Government Violation Data
CCTV

26. Complete Architecture Flow



27. Recommended Repository

```
nagpur-pulse/
│
├── frontend/
│
├── backend/
│
├── ml-service/
│
├── data/
│   ├── junctions/
│   ├── historical/
│   └── simulation/
│
├── docs/
│
└── README.md
```

28. Current Development Status

COMPLETED / AVAILABLE

- ☑ Project concept
- ☑ 40-junction dataset
- ☑ Historical accident dataset
- ☑ Police-unit simulation API
- ☑ Illegal-parking API
- ☑ TomTom API integration approach
- ☑ Routing/ETA architecture

IN DEVELOPMENT

- ☐ Traffic integration
- ☐ Incident integration
- ☐ Risk engine
- ☐ Heatmap

- ☐ Coverage engine
- ☐ Routing service
- ☐ Deployment optimization
- ☐ Recommendation system

FUTURE

- ☐ Official police data
- ☐ Police GPS
- ☐ CCTV
- ☐ Computer vision
- ☐ Advanced traffic prediction
- ☐ Production database

29. Developer Handover Summary

NAGPUR PULSE should be developed as a **modular decision-support platform** rather than a single monolithic application.

The most important separation is:

```
DATA
  |
  v
NORMALIZATION
  |
  v
RISK
  |
  v
COVERAGE
  |
  v
ROUTING
  |
  v
OPTIMIZATION
  |
  v
RECOMMENDATION
  |
  v
OPERATOR
```

External APIs and simulated datasets should always enter through adapters. This allows the current simulated police, parking, accident and event data to be replaced with authorized government data later without changing the core application.

The current prototype does **not** require MongoDB. MongoDB can be introduced later when persistent historical storage, audit logs, user management and production-scale data retention become necessary.

30. Quick Reference

Component	Current Technology
Frontend	React + TypeScript
UI	Tailwind CSS
Maps	Mapbox / Leaflet

Component	Current Technology
Backend	Node.js + Express
ML	Python + FastAPI
ML Models	XGBoost / Random Forest
Explainability	SHAP
Optimization	OR-Tools
Traffic	TomTom
Incidents	TomTom
Routing	TomTom
Geography	OpenStreetMap
Police Units	Beeceptor
Illegal Parking	Mockly
Historical Accidents	Uploaded XLSX
Current DB	Supabase/Neon ORM
Future DB	MongoDB
Real-time UI	Socket.IO
Future CV	YOLO + OpenCV